

UD3. MONGODB

Índice

Introducción.....	2
Docker y MongoDB.....	2
Ejecutar MongoDB como un contenedor Docker.....	2
Atlas.....	3
Creando un cluster.....	3
MongoDB Shell (mongosh).....	4
API de mongoDB.....	5
Listar y crear bases de datos.....	5
Crear una colección.....	5
Insertar un objeto.....	5
Insertar varios.....	6
Seleccionar (FIND).....	7
find().....	7
findOne().....	7
Filtro.....	7
Proyección.....	7
Update.....	7
updateOne().....	7
updateMany().....	8
Delete.....	8
deleteOne.....	8
deleteMany.....	8
Operadores de MongoDB.....	9
Comparación con match.....	9
Lógico.....	9
Evaluación.....	9
actualizar campos.....	9
actualizar matrices.....	9
Pipelines de agregación.....	10
Datos de muestra.....	10
Group.....	10
Limit.....	10
Project.....	10
Sort.....	11
match.....	11
count.....	11
Lookup.....	11

Introducción

MongoDB es una base de datos de documentos y puede instalarse localmente o alojarse en la nube.

Las bases de datos SQL se consideran bases de datos relacionales. Almacenan datos relacionados en tablas separadas. Cuando se necesitan datos, se consultan desde varias tablas para volver a unirlos.

MongoDB es una base de datos documental a la que a menudo se denomina base de datos no relacional o noSQL. Esto no significa que los datos relacionales no se puedan almacenar en bases de datos documentales, sino que los datos relacionales se almacenan de forma diferente. Una mejor forma de denominarla es como una base de datos no tabular.

MongoDB almacena datos en documentos flexibles. En lugar de tener varias tablas, puedes mantener todos los datos relacionados juntos. Esto hace que la lectura de los datos sea muy rápida.

También puedes tener varios grupos de datos. En MongoDB, en lugar de tablas, se denominan colecciones.

MongoDB se puede instalar localmente, lo que permitirá alojar servidor MongoDB propio en tu hardware. Esto requiere que administres el servidor, las actualizaciones y cualquier otro mantenimiento.

Docker y MongoDB

Existen muchas formas de utilizar MongoDB en un entorno de contenedores.

Ejecutar MongoDB como un contenedor Docker

Puedes iniciar un contenedor MongoDB con Docker con el siguiente comando.

```
docker run --name mongodb -d mongodb/mongodb-community-server:$MONGODB_VERSION
```

Si necesitas acceder al servidor MongoDB desde otra aplicación que se ejecuta localmente, deberás exponer un puerto utilizando el argumento -p.

```
docker run --name mongodb -d -p 27017:27017 mongodb-community-server:latest
```

Con este método, podrás conectarte a tu instancia de MongoDB en **mongodb://localhost:27017**. Puedes probarlo con **compass**, La GUI de MongoDB para visualizar y analizar datos.

Si deseas conservar los datos

```
docker run --name mongodb -d -p 27017:27017 -v $(pwd)/datos:/datos/db  
mongodb/mongodb-community-server:latest
```

Si usamos **–network nombre**. Te conectarás a MongoDB en **mongodb://nombre:27017** desde las otras aplicaciones en contenedores de la red.

```
docker run --name mongodb -d --network mongodb mongodb/mongodb-community-server:latest
```

Puedes probar esto con un segundo contenedor que ejecute **mongosh** en la misma red de forma interactiva o no

```
docker run --name mongosh --network mongodb mongodb/mongodb-community-server:latest mongosh mongodb://mongodb --eval "show dbs"
```

```
docker run -it mongodb/mongodb-community-server:latest mongosh "mongodb://nombreusuario:contraseña@clusterURL/base de datos"
```

Realmente podemos una vez creado un docker, ejecutarle un comando, por ejemplo, podemos crear un docker y luego ejecutar un comando con **exec**

```
docker exec -it nomeDoDocker comando
```

El comando puede ser **/bin/bash** para tener un terminal, **mongosh** para ejecutar el cliente de mongoDB etc.

Atlas

Puedes descargar y utilizar el *community-server* de código abierto de forma gratuita o incluso en un docker.

Sin embargo también vamos a utilizar MongoDB Atlas, una plataforma de base de datos en la nube. Esto es mucho más fácil que alojar tu propia base de datos local.

Con MongoDB Atlas, es fácil conectar todos los microservicios que componen tu sistema a la misma base de datos compartida. MongoDB Atlas también le ofrece todos los beneficios de ejecutar un clúster de MongoDB sin la molestia de configurar y administrar un conjunto de réplicas.

Para poder experimentar con los ejemplos de código, necesitarás acceso a una base de datos MongoDB.

Regístrate para obtener una cuenta gratuita de mongoDB Atlas para comenzar.

Creando un cluster

Después de haber creado tu cuenta, configura un "Clúster compartido" gratuito y luego elige tu proveedor de nube y región preferidos.

De forma predeterminada, MongoDB Atlas está completamente bloqueado y no tiene acceso externo.

Necesitarás configurar un usuario y agregar tu dirección IP a la lista de direcciones IP permitidas.

En “Acceso a la base de datos”, crea un nuevo usuario y guarda el nombre de usuario y la contraseña.

A continuación, en “Acceso a la red”, agrega tu dirección IP actual para permitir el acceso desde tu computadora.

En el botón “Conectar” puedes ver como conectarte con todos los lenguajes de programación, clientes, etc.

En el panel de MongoDB Atlas, en "Bases de datos", haz clic en el botón "Conectar" para tu clúster. A continuación, selecciona “Conectarse con MongoDB Shell”.

Copia tu cadena de conexión. Por ejemplo podría ser así

```
mongosh "mongodb+srv://cluster0.ex4ht.mongodb.net/myDatabase" --apiVersion 1 --username usuario
```

MongoDB Shell (mongosh)

Hay muchas formas de conectarse a su base de datos MongoDB.

Comenzaremos utilizando MongoDB Shell, mongosh. Puedes instalarlo como cualquier cliente en tu linux o usar un contenedor docker.

```
docker run --name mongodbServer -d -p 27017:27017 -v $(pwd)/datos:/datos/db  
mongodb/mongodb-community-server:latest
```

```
docker exec -it mongodbServer mongosh "mongodb+srv://....." --apiVersion 1 --username  
manuel --password lapassword
```

```
docker exec -it mongodbServer mongosh "localhost"
```

API de mongoDB

Listar y crear bases de datos

```
show dbs
```

```
use nombreDB
```

use crea la base de datos si no existe. Pero sólo lo hará si le creamos algo dentro.

Crear una colección

Podemos crear una colección con su propio comando `createCollection`

```
db.createCollection("posts")
```

O como parte de un insert, si insertamos un objeto en una colección inexistente la crea

Insertar un objeto

Inserta un único objeto dentro de una colección

```
db.posts.insertOne({  
  title: "Post Title 1",  
  body: "Body of post.",  
  category: "News",  
  likes: 1,  
  tags: ["news", "events"],  
  date: Date()  
})
```

Insertar varios

Inserta varios objetos de golpe, lo que hacemos es pasarle un array de objetos (Entre corchetes)

```
db.posts.insertMany( [  
  {  
    title: "Post Title 2",  
    body: "Body of post.",  
    category: "Event",  
    likes: 2,  
    tags: ["news", "events"],  
    date: Date()  
  },  
  {  
    title: "Post Title 3",  
    body: "Body of post.",  
    category: "Technology",  
    likes: 3,  
    tags: ["news", "events"],  
    date: Date()  
  },  
  {  
    title: "Post Title 4",  
    body: "Body of post.",  
    category: "Event",  
    likes: 4,  
    tags: ["news", "events"],  
    date: Date()  
  }  
])
```

Seleccionar (FIND)

Hay dos métodos para seleccionar datos de una colección : `find()` y `findOne()`

`find()`

Para seleccionar datos en una colección en MongoDB usamos `find` que acepta un objeto que hace de filtro. Si el objeto está vacío devolverá todos los documentos:

```
db.posts.find()
```

`findOne()`

Funciona igual que el anterior pero si no le pasamos un objeto filtro devuelve sólo el primer documento

```
db.posts.findOne()
```

Filtro

El objeto filtro no tiene por que ser completo:

```
db.posts.find( {category: "News"} )
```

devolvería todos los documentos (objetos) que sean de la categoría News

Proyección

Podemos pasarle un segundo parámetro que sea la proyección, es decir, que atributos del objeto queremos que nos devuelva. Para ello le pasamos un objeto con los atributos con un 1 si queremos que salgan, un 0 si no queremos que salgan, pero ojo, no pueden mezclarse salvo si queremos excluir el identificador de la fila `_id`

```
db.posts.find({}, { title: 1, date: 1 })
```

```
db.posts.find({}, { category: 0 })
```

```
db.posts.find({}, { _id: 0, title: 1, date: 1 })
```

```
db.posts.find({}, { title: 1, date: 0 }) -- esto está mal
```

Update

Para actualizar un documento podemos usar `updateOne()` o `updateMany()` .

El primer parámetro es un objeto filtro para definir que documento o documentos serán actualizados.
El segundo parámetro es un objeto definiendo los nuevos valores de los datos.

`updateOne()`

Actualizará el primer documento que encuentra el filtro

```
db.posts.updateOne( { title: "Post Title 1" }, { $set: { likes: 2 } } )
```

```
db.posts.updateOne( { title: "Post Title 1" }, { $inc: { likes: 1 } } ) --equivalente a likes++
```

Si queremos que se inserte si no existe podemos usar **upsert**

```
db.posts.updateOne(
  { title: "Post Title 5" },
  {
    $set:
    {
      title: "Post Title 5",
      body: "Body of post.",
      category: "Event",
      likes: 5,
      tags: ["news", "events"],
      date: Date()
    }
  },
  { upsert: true }
)
```

updateMany()

Actualiza todos los documentos devueltos por el filtro

```
db.posts.updateMany({}, { $inc: { likes: 1 } })
```

Delete

El borrado también existe con dos comandos:

deleteOne

```
db.posts.deleteOne({ title: "Post Title 5" })
```

deleteMany

```
db.posts.deleteMany({ category: "Technology" })
```

Operadores de MongoDB

Hay muchos operadores de consulta que se pueden utilizar para comparar y hacer referencia a campos de documentos.

Comparación con match

\$eq : Los valores son iguales

\$ne : Los valores no son iguales

\$gt : El valor es mayor que otro valor

\$gte : El valor es mayor o igual a otro valor

\$lt : El valor es menor que otro valor

\$lte : El valor es menor o igual a otro valor

\$in : El valor coincide dentro de una matriz

Lógico

\$and: Devuelve documentos donde coinciden ambas consultas

\$or: Devuelve documentos en los que coincide alguna de las consultas

\$nor: Devuelve documentos en los que ambas consultas no coinciden

\$not: Devuelve documentos donde la consulta no coincide

Evaluación

\$regex: Permite el uso de expresiones regulares al evaluar valores de campo.

\$text: Realiza una búsqueda de texto

\$where: Utiliza una expresión de JavaScript para hacer coincidir documentos

actualizar campos:

\$currentDate: Establece el valor del campo en la fecha actual

\$inc: Incrementa el valor del campo

\$rename: Cambia el nombre del campo

\$set: Establece el valor de un campo

\$unset: Elimina el campo del documento

actualizar matrices.

\$addToSet: Agrega elementos distintos a una matriz

\$pop: Elimina el primer o último elemento de una matriz

\$pull: Elimina todos los elementos de una matriz que coinciden con la consulta

\$push: Agrega un elemento a una matriz

Pipelines de agregación

Las operaciones de agregación permiten agrupar, ordenar, realizar cálculos, analizar datos y mucho más. Se ponen entre corchetes pues es como un array de etapas.

Los canales de agregación pueden tener una o más "etapas". El orden de estas etapas es importante. Cada etapa actúa sobre los resultados de la etapa anterior.

```
db.posts.aggregate( [  
  // Etapa 1: Sólo devuelve documentos que tengan más de 1 like  
  { $match: { likes: { $gt: 1 } } },  
  // Etapa 2: Sobre los documentos anteriores agrupa por categoría y suma los likes  
  { $group: { _id: "$category", totalLikes: { $sum: "$likes" } } }  
])
```

Datos de muestra

Para demostrar el uso de etapas en una cadena de agregación, cargaremos datos de muestra en nuestra base de datos.

Desde el panel de control de MongoDB Atlas, vete a "Bases de datos". Haz clic en los puntos suspensivos y selecciona "Cargar conjunto de datos de muestra". Esto cargará varios conjuntos de datos de muestra en tu base de datos.

Group

Agrupar por el atributo que queramos que pongamos en el `_id`. Usando `sample_airbnb`

```
db.listingsAndReviews.aggregate([ { $group : { _id : "$property_type" } } ])
```

Limit

Limita el número de documentos pasados a la siguiente etapa. Usando la base de datos `sample_mflix`

```
db.movies.aggregate([ { $limit: 1 } ])
```

Project

Es la misma proyección que usamos en `find()`. Usando `sample_restaurants`

```
db.restaurants.aggregate([  
  { $project: { "name": 1, "cuisine": 1, "address": 1 } }, { $limit: 5 }  
])
```

Sort

Ordena, es como un order by. **1 ordena ascendente y -1 ordena descendente**

```
db.listingsAndReviews.aggregate([
  { $sort: { "accommodates": -1 } }, { $project: { "name": 1, "accommodates": 1 } }, { $limit: 5 }
])
```

match

Filtra los documentos que serán pasados a la siguiente etapa

```
db.listingsAndReviews.aggregate([
  { $match : { property_type : "House" } },
  { $limit: 2 },
  { $project: {
    "name": 1,
    "bedrooms": 1,
    "price": 1
  }}
])
```

count

Cuenta los documentos pasados desde la etapa anterior

```
db.restaurants.aggregate([
  { $match: { "cuisine": "Chinese" } }, { $count: "totalChinese" }
])
```

Lookup

Esta etapa de agregación realiza un left outer join a una colección en la misma base de datos.

Hay cuatro campos obligatorios:

from: La colección que se utilizará para la búsqueda en la misma base de datos

localField: El campo de la colección principal por el que unimos

foreignField: El campo de la otra colección (en relacional sería PK)

as: El nombre del nuevo campo que contendrá los documentos coincidentes.

```
db.comments.aggregate([
  {
    $lookup: {
      from: "movies",
      localField: "movie_id",
      foreignField: "_id",
      as: "movie_details",
    }, }, { $limit: 1 }
])
```